

BASH SCRIPTING FUNDAMENTALS

Text Processing

grep, sed, awk, cut, sort, and practical pipelines

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026

July 2026



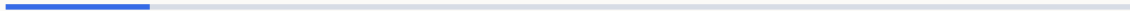
Agenda

- 1 **Section 1: grep Essentials** — 4 slides
- 2 **Section 2: grep Patterns** — 4 slides
- 3 **Section 3: Selecting and Translating Text** — 4 slides
- 4 **Section 4: Sorting and Deduplicating** — 4 slides
- 5 **Section 5: Combining and Formatting Lines** — 4 slides
- 6 **Section 6: sed Basics** — 4 slides
- 7 **Section 7: awk Foundations** — 4 slides
- 8 **Section 8: awk Practice** — 4 slides

- 9 **Section 9: Practical Pipelines** — 4 slides
- 1 **Section 10: Watching and Boundaries** — 4 slides
- 0
- 1 **Section 11: Mistakes and Checkpoints** — 4 slides
- 1

1 Section 1

grep Essentials



1.1 grep finds matching lines · grep -n prints line numbers

- **grep** searches text and prints lines that match.
- The pattern comes before the file names.
- With no file name, **grep** reads stdin.
- -n prefixes each match with its line number.
- Line numbers help you jump back into files.
- **They** are useful in logs and source code searches.

Examples

```
# Search a file
grep ERROR app.log
# Search stdin
printf 'ok\nerror\n' | grep error
# Line numbers
grep -n TODO script.sh
# Numbered stdin
printf 'a\nb\n' | grep -n b
```

1.2 grep -i ignores case · grep -v inverts matches

- -i matches uppercase and lowercase forms.
- **It** is useful when input capitalization is inconsistent.
- Use exact case when capitalization is meaningful.
- -v prints lines that do not match.
- It removes noise from a stream.
- Be careful not to hide lines you still need.

Examples

```
# Case-insensitive log search
grep -i error app.log
# Case-insensitive word
printf 'Root\nuser\n' | grep -i root
# Drop comments
grep -v '^#' config.ini
# Drop blank marker
printf 'ok\nskip\n' | grep -v skip
```

1.3 grep -w matches words · grep -c counts matching lines

- -w requires the match to be a whole word.
- It avoids matching substrings inside longer names.
- Word boundaries follow **grep**'s definition of word characters.
- -c prints the number of matching lines.
- It counts lines, not individual matches.
- Use it for quick health checks and summaries.

Examples

```
# Whole word user
grep -w user accounts.txt
# Avoid substring
printf 'cat\nscatter\n' | grep -w cat
# Count errors
grep -c ERROR app.log
# Count stdin matches
printf 'x\ny\nx\n' | grep -c x
```

1.4 grep -l lists matching files · grep -R searches recursively

- -l prints file names instead of matching lines.
- It stops after the first match in each file.
- Use it to find which files need attention.
- -r or -R searches through directories.
- Recursive search can be noisy in large trees.
- Limit the path when you know where to look.

Examples

```
# Files with TODO
grep -l TODO *.sh
# Configs with port
grep -l 'port=' configs/*
# Search scripts
grep -R TODO scripts/
# Search current tree
grep -R 'set -e' .
```


2 Section 2

grep Patterns



2.1 grep -E enables extended regular expressions · grep -F searches fixed strings

- -E supports clearer alternation and grouping syntax.
- **It** is the same family as egrep on many systems.
- Quote regex patterns so the shell does not change them.
- -F treats the pattern as literal text.
- It avoids regex meaning for characters like . and *.
- Fixed searches are often faster and safer for user text.

Examples

```
# Alternation
grep -E 'ERROR|WARN' app.log
# Grouped suffix
grep -E 'file\.(txt|csv)$' list.txt
# Literal dots
grep -F '10.0.0.1' access.log
# Literal brackets
grep -F '[prod]' app.log
```

2.2 grep context shows nearby lines · Anchors match line positions

- -A prints lines after a match.
- -B prints lines before a match.
- -C prints lines on both sides.
- ^ matches the start of a line.
- \$ matches the end of a line.
- **Anchors** make patterns more precise.

Examples

```
# After context
grep -A 2 ERROR app.log
# Both sides
grep -C 3 timeout app.log
# Starts with error
grep '^ERROR' app.log
# Ends with done
grep 'done$' tasks.txt
```

2.3 Character classes describe sets · Dot and star can overmatch

- `[0-9]` matches one digit in many grep modes.
- `[:digit:]` is a named **character** class.
- **Classes** are clearer than listing every character.
- `.` matches almost any single **character**.
- `*` repeats the previous pattern zero or more times.
- Prefer specific patterns when extracting data.

Examples

```
# Any digit
grep '[0-9]' ids.txt
# Named class
grep '[:digit:]' ids.txt
# Broad match
grep 'user=.*' config.ini
# Specific match
grep 'user=[^ ]*' config.ini
```

2.4 Quote grep patterns · ripgrep is a fast optional searcher

- Shell globbing can change unquoted patterns.
- Single quotes preserve regex characters for grep.
- Use double quotes only when shell variables must expand.
- **rg** is a modern recursive search tool.
- It respects many ignore files by default.
- Use grep when portability matters most.

Examples

```
# Quoted regex
grep '^[A-Z]' names.txt
# Variable pattern
pat='ERROR'
grep "$pat" app.log
# Search with rg
rg TODO scripts/
# Line numbers
rg -n 'set -e' scripts/
```

3 Section 3

Selecting and Translating Text

3.1 cut -d -f selects delimited fields · cut -c selects character positions

- -d chooses the field delimiter.
- -f chooses one or more fields.
- **cut** is simple and fast for regular delimited data.
- -c extracts by character position.
- Ranges like 1-8 select spans.
- It works best on fixed-width text.

Examples

```
# First colon field
cut -d: -f1 /etc/passwd
# Second CSV field
cut -d, -f2 users.csv
# First eight chars
cut -c1-8 dates.txt
# Single character
printf 'abc\n' | cut -c2
```

3.2 tr -d deletes characters · tr -s squeezes repeated characters

- **tr** reads stdin and writes translated stdout.
- -d removes listed characters.
- **It** is useful for simple cleanup steps.
- -s replaces repeated listed characters with one.
- **It** is common for normalizing spaces.
- Squeeze before cutting when spacing is irregular.

Examples

```
# Remove carriage returns
tr -d '\r' < win.txt > unix.txt
# Remove digits
printf 'a1b2\n' | tr -d '0-9'
# Squeeze spaces
printf 'a  b\n' | tr -s ' '
# Squeeze blank lines
tr -s '\n' < text.txt
```


3.3 tr translates characters · wc counts lines, words, and bytes

- **tr** maps characters from one set to another.
- The first set and second set are positional.
- Use it for simple case or delimiter changes.
- -l counts lines.
- -w counts words.
- -c counts bytes.

Examples

```
# Uppercase
printf 'abc\n' | tr '[:lower:]' '[:upper:]'
# Commas to tabs
tr ',' '\t' < data.csv
# Count lines
wc -l app.log
# Count all three
wc -l -w -c README.md
```

3.4 head shows the beginning · tail shows the end

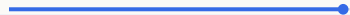
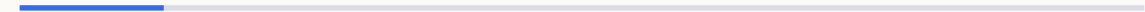
- **head** prints the first lines of input.
- -n controls how many lines to show.
- **It** is useful for checking file shape before processing.
- **tail** prints the last lines of input.
- -n controls how many lines to show.
- **It** is often used for recent log entries.

Examples

```
# First ten lines
head app.log
# First three lines
head -n 3 app.log
# Last ten lines
tail app.log
# Last five lines
tail -n 5 app.log
```

4 Section 4

Sorting and Deduplicating



4.1 sort orders lines · sort -n orders numbers

- **sort** reads lines and writes them in order.
- Default **sort** order is lexicographic.
- **Sorting** is often required before **uniq**.
- **-n** compares numeric values.
- Without **-n**, 10 can **sort** before 2.
- Use numeric **sort** for counts, sizes, and IDs.

Examples

```
# Sort names
sort names.txt
# Sort stdin
printf 'b\na\n' | sort
# Numeric order
printf '10\n2\n' | sort -n
# Sort counts
sort -n counts.txt
```

4.2 sort -r reverses order · sort -k chooses a key

- -r reverses the final **sort** order.
- Combine it with -n for largest numbers first.
- Reverse **sort** is common for leaderboards.
- -k selects which field to **sort** by.
- Field splitting defaults to runs of blanks.
- Use -t when the delimiter is not whitespace.

Examples

```
# Reverse text
sort -r names.txt
# Largest first
sort -nr counts.txt
# Sort by second field
sort -k2 users.txt
# Sort CSV by third field
sort -t, -k3 users.csv
```

4.3 sort -u sorts unique lines · uniq removes adjacent duplicates

- -u removes duplicate lines while sorting.
- **It** is a compact sort plus unique operation.
- Use it when duplicate order does not matter.
- **uniq** compares neighboring lines only.
- **Sort** first when duplicates may be separated.
- It preserves the first line of each adjacent run.

Examples

```
# Unique sorted names
sort -u names.txt
# Unique from stdin
printf 'b\na\nb\n' | sort -u
# Adjacent duplicates
uniq sorted.txt
# Sort then uniq
sort names.txt | uniq
```

4.4 uniq -c counts duplicate runs · uniq -d prints duplicated lines

- -c prefixes each output line with a count.
- It counts adjacent runs, so sorted input matters.
- **This** is a core frequency-counting pattern.
- -d prints only lines that repeat in adjacent runs.
- Sort first to find duplicates anywhere in a file.
- It helps detect repeated IDs or records.

Examples

```
# Count names
sort names.txt | uniq -c
# Count statuses
cut -d' ' -f1 app.log | sort | uniq -c
# Find duplicate names
sort names.txt | uniq -d
# Duplicate IDs
cut -d, -f1 users.csv | sort | uniq -d
```

5 Section 5

Combining and Formatting Lines

5.1 paste combines lines side by side · join merges sorted files on a key

- **paste** joins corresponding lines from files.
- The default delimiter is a tab.
- `-d` chooses another delimiter.
- **join** combines lines that share a field.
- Input files must be sorted on the **join** field.
- **It** is useful for simple table lookups.

Examples

```
# Paste two files
paste names.txt scores.txt
# Comma delimiter
paste -d, names.txt scores.txt
# Join by first field
join users.sorted scores.sorted
# Join CSV-like data
join -t, users.csv scores.csv
```

5.2 column -t aligns tables · rev reverses characters on each line

- **column** -t formats columns into a readable table.
- **It** is for display, not reliable data storage.
- -s chooses the input delimiter.
- **rev** reverses each line independently.
- It can help with suffix-oriented tricks.
- Use clearer tools when the intent is not obvious.

Examples

```
# Align whitespace table
column -t report.txt
# Align CSV display
column -s, -t users.csv
# Reverse text
printf 'abc\n' | rev
# Look at suffixes
rev files.txt | sort | rev
```

5.3 fmt refills paragraphs · fold wraps long lines

- **fmt** wraps text into paragraphs.
- **It** is useful for plain prose.
- **It** is not meant for structured logs or CSV.
- **fold** breaks long lines at a width.
- -w chooses the output width.
- -s tries to break at spaces.

Examples

```
# Wrap prose
fmt notes.txt
# Set width
fmt -w 60 notes.txt
# Fold at 40
fold -w 40 long.txt
# Fold on spaces
fold -s -w 72 paragraph.txt
```

5.4 expand converts tabs to spaces · unexpand converts spaces to tabs

- **expand** replaces tab characters with spaces.
- -t controls tab stops.
- It helps normalize text for display.
- **unexpand** replaces runs of spaces with tabs.
- -a can convert beyond leading blanks.
- Use it only when **tabs** are desired by the target format.

Examples

```
# Default expand
expand table.txt
# Four-space tabs
expand -t 4 Makefile
# Leading blanks
unexpand indented.txt
# All blanks
unexpand -a aligned.txt
```

6 Section 6

sed Basics



6.1 sed s replaces the first match · sed s with g replaces all matches

- s/pattern/replacement/ substitutes text.
- By default, **sed** replaces the first match on each line.
- **sed** writes changed text to stdout unless told otherwise.
- The g flag means global within each line.
- Use it when repeated matches on a line should change.
- Without g, only the first match changes.

Examples

```
# Replace first match
sed 's/error/ERROR/' app.log
# Replace delimiter text
printf 'a:b\n' | sed 's/:/,/'
# Replace all spaces
sed 's/ /_/g' names.txt
# Replace all commas
printf 'a,b,c\n' | sed 's/,/|/g'
```

6.2 sed -n suppresses automatic printing · sed can print ranges

- **sed** normally prints every processed line.
- -n stops that automatic output.
- Combine -n with p to print selected lines.
- Address ranges select a span of lines.
- A range can use line numbers or patterns.
- Use -n to print only the selected range.

Examples

```
# Print matching lines
sed -n '/ERROR/p' app.log
# Print line two
sed -n '2p' file.txt
# Lines two to five
sed -n '2,5p' file.txt
# Between markers
sed -n '/BEGIN/,/END/p' file.txt
```

6.3 sed d deletes selected lines · sed line addresses target positions

- The d command removes matching lines from output.
- It does not modify the original file by default.
- Use it to filter streams by line address.
- A number selects one line.
- \$ selects the last line.
- Addresses can combine with commands like p or d.

Examples

```
# Delete blanks
sed '/^$/d' notes.txt
# Delete comments
sed '/^#/d' config.ini
# Delete first line
sed '1d' data.csv
# Print last line
sed -n '$p' data.csv
```


6.4 sed -i edits files in place · sed can use alternate delimiters

- -i changes the file instead of only stdout.
- Test without -i before modifying important files.
- Use a backup suffix when available and useful.
- The delimiter after s does not have to be slash.
- Alternate delimiters reduce escaping in paths and URLs.
- Pick a delimiter absent from the pattern and replacement.

Examples

```
# Preview change
sed 's/dev/prod/' app.conf
# In-place with backup
sed -i.bak 's/dev/prod/' app.conf
# Path replacement
sed 's#/tmp#/var/tmp#g' paths.txt
# URL replacement
sed 's|http://|https://|g' urls.txt
```

7 Section 7

awk Foundations

7.1 awk fields use \$1, \$2, and more · awk -F sets the field separator

- **awk** splits each input line into fields.
- \$1 is the first field and \$2 is the second field.
- \$0 is the whole original line.
- -F chooses how **awk** splits fields.
- Use -F: for colon files and -F, for simple CSV.
- The **separator** is a regular expression.

Examples

```
# Print first field
awk '{print $1}' users.txt
# Print whole line
awk '{print $0}' users.txt
# Colon field
awk -F: '{print $1}' /etc/passwd
# CSV field
awk -F, '{print $2}' users.csv
```

7.2 awk NR counts records · awk NF counts fields

- **NR** is the current input record number.
- For normal text, a record is usually a line.
- **NR** is useful for skipping headers and printing positions.
- **NF** is the number of fields on the current record.
- \$NF is the last field.
- NF helps find malformed or variable-width lines.

Examples

```
# Number lines
awk '{print NR, $0}' file.txt
# Skip header
awk 'NR>1 {print $0}' data.csv
# Print field count
awk '{print NF, $0}' file.txt
# Print last field
awk '{print $NF}' file.txt
```

7.3 awk patterns select records · awk BEGIN runs before input

- A pattern before braces decides whether the action runs.
- Regex patterns use slashes.
- Numeric and string comparisons are also patterns.
- BEGIN actions run before the first record.
- Use BEGIN to print headers or set variables.
- BEGIN does not require any input records.

Examples

```
# Regex pattern
awk '/ERROR/ {print $0}' app.log
# Numeric pattern
awk '$3 > 80 {print $1}' scores.txt
# Print header
awk 'BEGIN {print "name score"} {print $1, $2}' scores.txt
# Set separator
awk 'BEGIN {FS=":"} {print $1}' /etc/passwd
```

7.4 awk END runs after input · awk print builds output records

- END actions run after all records are read.
- Use END for totals and summaries.
- Variables accumulated during input are available in END.
- print joins arguments with the output field separator.
- Commas in print are not literal commas.
- Set OFS when downstream tools expect a delimiter.

Examples

```
# Count records
awk 'END {print NR}' app.log
# Print final total
awk '{sum+=$1} END {print sum}' nums.txt
# Print two fields
awk '{print $1, $3}' users.txt
# Comma output
awk 'BEGIN {OFS=","} {print $1,$3}' users.txt
```

8 Section 8

awk Practice

8.1 awk can sum a column · awk can compute averages

- Add field values to a variable for each record.
- Print the accumulated value in END.
- Numeric conversion happens automatically for numeric text.
- Keep a running sum and count.
- Use END after all input has been processed.
- Guard against dividing by zero in reusable scripts.

Examples

```
# Sum first column
awk '{sum += $1} END {print sum}' nums.txt
# Sum CSV amount
awk -F, '{sum += $3} END {print sum}' sales.csv
# Average first column
awk '{sum+=$1} END {print sum/NR}' nums.txt
# Average score
awk 'NR>1 {sum+=$2; n++} END {print sum/n}' scores.txt
```


8.2 awk can count matching records · awk extracts and rearranges columns

- Increment a variable only on selected records.
- A regex pattern is enough for simple log counts.
- Print the counter in END.
- Print fields in the order you need.
- Set FS for input and OFS for output.
- **This** is clearer than complex cut pipelines for rearranging.

Examples

```
# Count errors
awk '/ERROR/ {n++} END {print n}' app.log
# Count status code
awk '$9 == 500 {n++} END {print n}' access.log
# Swap fields
awk '{print $2, $1}' names.txt
# CSV subset
awk -F, 'BEGIN{OFS=","} {print $3,$1}' users.csv
```

8.3 awk arrays group counts · awk conditions filter records

- Associative arrays use strings as keys.
- Increment an array element for each record's group.
- Print grouped counts in END.
- Comparisons can use numbers or strings.
- Logical operators combine conditions.
- Keep conditions readable by splitting complex pipelines.

Examples

```
# Count first field
awk '{count[$1]++} END {for (k in count) print k, count[k]}' data.txt

# Count CSV status
awk -F, '{count[$2]++} END {for (k in count) print k, count[k]}' data.csv

# High scores
awk '$2 >= 90 {print $1}' scores.txt

# Two checks
awk '$3 == "prod" && $4 > 0 {print $1}' services.txt
```

8.4 awk split breaks a field apart · awk -v passes shell values safely

- `split(string, array, separator)` divides text.
- The **pieces** are stored starting at index 1.
- Use it when only one field has a nested delimiter.
- `-v` sets an **awk** variable before processing begins.
- It avoids fragile quote mixing inside the program.
- Quote shell variables when passing them.

Examples

```
# Split date
awk '{split($1,d,"-"); print d[1]}' dates.txt
# Split email
awk -F, '{split($2,a,"@"); print a[1]}' users.csv
# Pass threshold
limit=80
awk -v limit="$limit" '$2 > limit {print $1}' scores
name=alice
awk -v name="$name" '$1 == name {print $0}' users.txt
```

9 Section 9

Practical Pipelines

9.1 Combine grep, cut, sort, and uniq · Log parsing starts with stable fields

- Filter first to reduce the data stream.
- Extract only the field you need.
- Sort before counting with `uniq -c`.
- Know the **log** format before choosing field numbers.
- Use `grep` to narrow lines and `awk` to select fields.
- Test on a small sample with `head`.

Examples

```
# Count error codes
grep ERROR app.log | cut -d' ' -f3 | sort | uniq -c
# Count users
cut -d, -f1 users.csv | sort | uniq -c
# Sample logs
head -n 5 access.log
# Extract status
awk '{print $9}' access.log | sort | uniq -c
```

9.2 Counting errors is a common pipeline · Extract columns after confirming delimiters

- Choose the error marker precisely.
- Count matching lines with `grep -c` for simple cases.
- Use grouped counts when errors have categories.
- **cut** is best for simple single-character delimiters.
- **awk** is better for whitespace and rearranging fields.
- CSV with quotes needs a CSV-aware tool.

Examples

```
# Simple error count
grep -c ERROR app.log
# Error categories
grep ERROR app.log | awk '{print $3}' | sort | uniq
# Simple CSV column
cut -d, -f2 users.csv
# Whitespace column
awk '{print $2}' table.txt
```

9.3 Top values need counts and numeric sort · Whitespace cleanup normalizes input

- Count values with sort and uniq -c.
- Sort counts numerically in reverse order.
- Use head to keep the largest entries.
- Trim or squeeze spaces before field extraction when needed.
- tr -s handles repeated single characters.
- sed can remove leading or trailing **whitespace**.

Examples

```
# Top paths
awk '{print $7}' access.log | sort | uniq -c | sort -nr
# Top users
cut -d, -f1 events.csv | sort | uniq -c | sort -nr
# Squeeze spaces
tr -s ' ' < messy.txt
# Trim trailing spaces
sed 's/[[[:space:]]\+$// ' messy.txt
```

9.4 dos2unix removes Windows line endings · Encoding affects text tools

- Windows text may end lines with carriage return plus newline. **Examples**
- The carriage return can break comparisons and scripts.
- `tr -d '\r'` is a simple portable concept.
- Text tools operate on bytes and characters according to locale.
- Unexpected encodings can break sorting and matching.
- Normalize input when pipelines produce surprising results.

```
# Remove carriage returns
tr -d '\r' < win.txt > unix.txt
# Detect carriage returns
grep '$\r' win.txt
# Show locale
locale
# Byte-oriented sort
LC_ALL=C sort names.txt
```


10

Section 10

Watching and Boundaries

10.1 tail -f follows growing files · head and tail sample safely

- -f keeps reading as new lines are appended.
- **It** is useful for live log watching.
- Pipe it into grep for focused monitoring.
- **head** checks the beginning of a large file.
- **tail** checks recent or ending lines.
- Sampling helps avoid running heavy pipelines blindly.

Examples

```
# Follow a log
tail -f app.log
# Follow errors
tail -f app.log | grep ERROR
# Check shape
head -n 3 data.csv
# Check ending
tail -n 3 data.csv
```

10.2 Words and substrings differ · Locale changes sort order

- A substring match can find text inside longer values.
- `grep -w` limits matches to **words**.
- Choose based on whether partial matches are valid.
- `sort` follows **locale** collation rules by default.
- `LC_ALL=C` gives byte-oriented order.
- Use a fixed **locale** for reproducible scripts.

Examples

```
# Substring
printf 'cat\nscatter\n' | grep cat
# Whole word
printf 'cat\nscatter\n' | grep -w cat
# Default sort
sort names.txt
# C locale sort
LC_ALL=C sort names.txt
```

10.3 Tabs and spaces are different delimiters · Simple CSV is not all CSV

- A visual gap may be spaces, **tabs**, or both.
- `cut -f` defaults to tab-delimited fields.
- Use `tr` or `awk` when spacing is inconsistent.
- `cut -d,` works only for **simple** comma-separated data.
- Quoted commas require a real CSV parser.
- Be honest about the data format before scripting.

Examples

```
# Tab fields
cut -f1 table.tsv
# Show tabs visibly
sed -n '1' table.tsv
# Simple field
cut -d, -f3 users.csv
# Awk simple CSV
awk -F, '{print $3}' users.csv
```

10.4 Text tools expect text · Choose the smallest clear tool

- Binary input can produce strange terminal output.
- grep may report binary file matches.
- Use file or strings before treating unknown data as **text**.
- grep filters lines.
- cut extracts simple fields.
- awk handles field logic and summaries.

Examples

```
# Identify file type
file archive.bin
# Extract printable text
strings archive.bin | head
# Line filter
grep ERROR app.log
# Field summary
awk '{count[$1]++} END {for (k in count) print k,count[k]}
```

11

Section 11

Mistakes and Checkpoints

11.1 Common grep mistake: unquoted patterns · Common cut mistake: wrong delimiter

- Unquoted patterns can be expanded by the shell.
- A pattern that works in one directory may fail in another.
- Quote patterns unless you intentionally need shell expansion.
- cut does not guess delimiters.
- A comma delimiter will not split tab-separated data.
- Inspect a sample before choosing -d.

Examples

```
# Risky pattern
grep *.log files.txt
# Quoted pattern
grep '*.log' files.txt
# Inspect sample
head -n 1 data.txt
# Choose delimiter
cut -d: -f1 /etc/passwd
```

11.2 Common uniq mistake: unsorted input · Common sed mistake: editing too soon

- `uniq` only removes adjacent duplicates.
- Separated duplicates remain unless input is sorted.
- Use `sort | uniq` for global duplicate removal.
- `sed -i` changes files immediately.
- Preview substitutions on stdout first.
- Use backups for important one-off edits.

Examples

```
# Wrong for separated duplicates
uniq names.txt
# Global unique lines
sort names.txt | uniq
# Preview first
sed 's/old/new/g' file.txt
# Then edit with backup
sed -i.bak 's/old/new/g' file.txt
```


11.3 Common awk mistake: shell quoting · Practice pipelines one stage at a time

- awk programs often contain \$ characters.
- Single quotes protect \$1 from shell expansion.
- Use -v to pass shell values into awk.
- Run the first command and inspect its output.
- Add one stage only after the previous stage is correct.
- Keep intermediate commands readable in shell history.

Examples

```
# Protect fields
awk '{print $1}' users.txt
# Pass a value
field=2
awk -v f="$field" '{print $f}' users.txt
# Start simple
grep ERROR app.log
# Add stages
grep ERROR app.log | awk '{print $3}' | sort | uniq
```

11.4 Review text processing choices · Module 06 checkpoint

- Use grep for lines, cut for simple columns, sed for edits.
- Use sort and uniq for ordering and counts.
- Use awk when field logic becomes conditional or numeric.
- Count a category from a log file.
- Extract and sort one column from delimited text.
- Use sed or awk to clean one messy input stream.

Examples

```
# Line and column
grep active users.csv | cut -d, -f1
# Numeric summary
awk '{sum+=$2} END {print sum}' scores.txt
# Count categories
awk '{print $3}' app.log | sort | uniq -c
# Clean and extract
tr -s ' ' < table.txt | cut -d' ' -f2
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
06 · Text Processing

